

KAIRO PRODUCTION TOOLS

A multi-DCC artist pipeline from Python validation to C++ engine ingest.

4 creative applications

Blender, Houdini, Nuke, Maya

1 production contract

diagnose -> dry-run -> publish -> verify

2 language layers

Python artist tools + C++23 engine integration

Swayam Singal

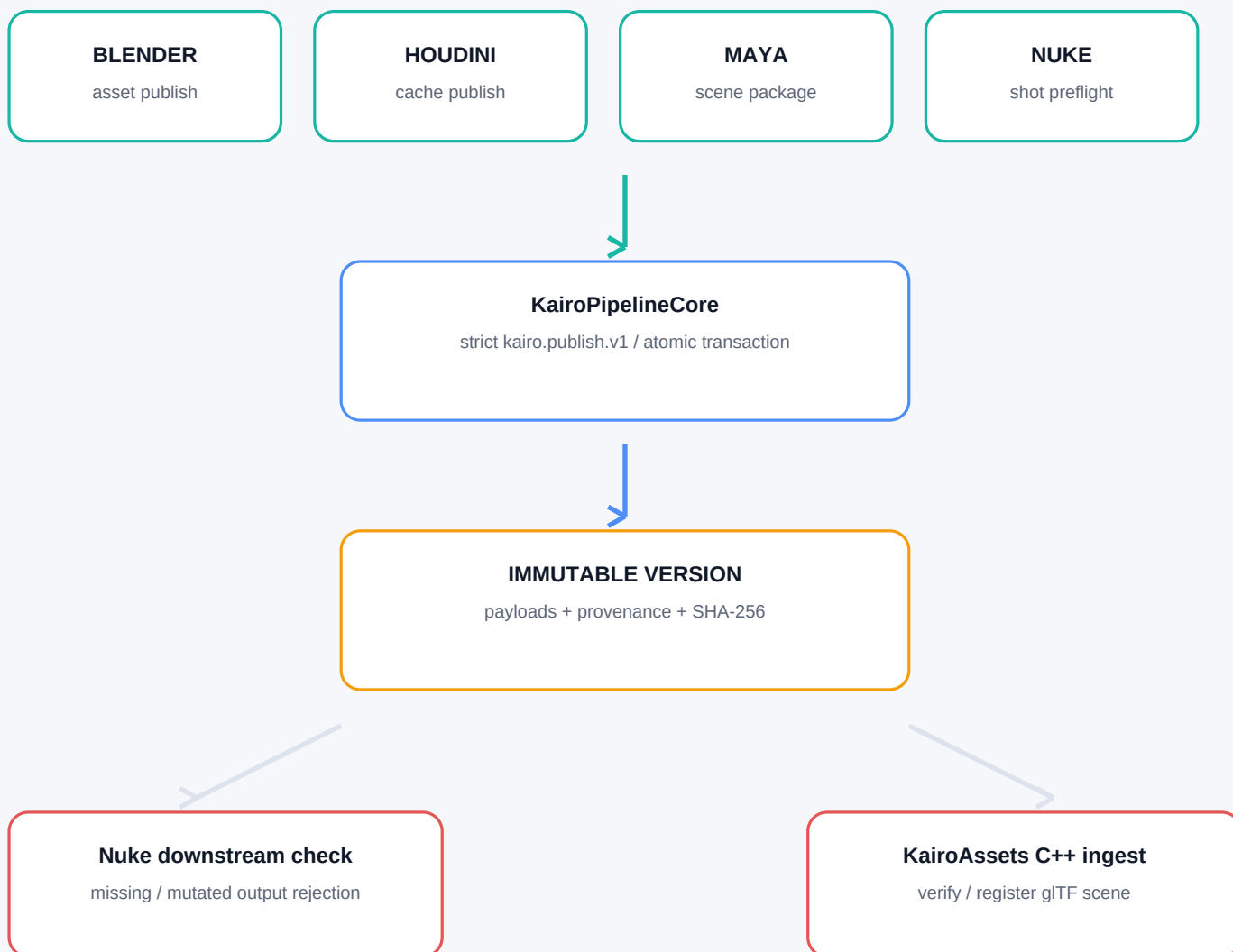
Graphics / engine programmer extending into production technical direction

github.com/swayam8624/KairoProductionTools

THE SYSTEM

One contract, many creative hosts

Each application keeps its native workflow. The shared layer standardises diagnostics, portable paths, fingerprints, frame sequences, and immutable publication.



Artist feedback

Errors carry stable codes, severity, suggestions, and a host location for navigation.

Safe mutation

Dry-run creates nothing. Publication becomes visible only after full staging and verification.

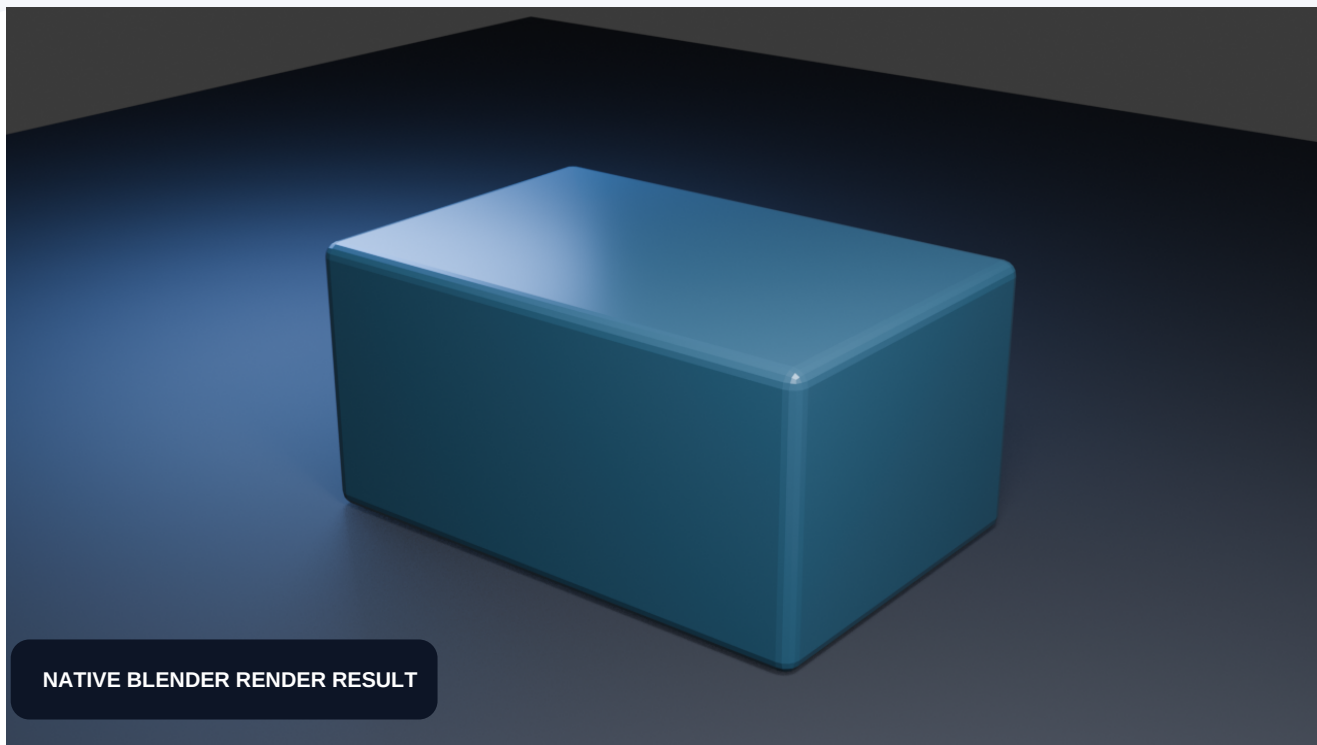
Downstream trust

Consumers recalculate fingerprints instead of trusting timestamps or filenames.

PROJECT 01 / NATIVE VERIFIED

Blender to Kairo asset pipeline

An artist can find a production problem, navigate to it, apply a bounded fix, and publish a real glTF package without leaving Blender.



VALIDATE

NAVIGATE

SAFE FIX

DRY-RUN

PUBLISH

Artist surface

Blender panel, diagnostics list, property navigation, scale fix, selected/all scope, version controls.

Production output

Separate glTF, binary buffer, textures, source provenance, per-file SHA-256, atomic version directory.

Evidence

Blender 5.2 LTS: 4 native tests, 4 pure tests, fresh-profile install, package validation, 1280x720 render.

[GitHub: KairoBlender](#)

SHARED INFRASTRUCTURE

A handoff that can prove what it consumed

The engine does not trust that a DCC publish stayed unchanged. Python records the contract; C++ reloads it strictly and verifies every byte before registry mutation.

PYTHON PUBLICATION

```
manifest = PublishManifest(  
    kind=ASSET, source_host='blender',  
    outputs=(scene,), dependencies=files)  
  
plan_publish(...)      # no writes  
publish_bundle(...)     # stage + verify  
os.replace(staging, target)
```

C++23 INGESTION

```
auto manifest = LoadPipelinePublishManifest(path);  
ValidatePipelinePublishBundle(path, manifest);  
  
AssetID id = RegisterBlenderPublish(  
    projectRoot, path, registry);  
  
// rejects tamper / escape / symlink
```

Portable schema

Exact keys and types, bounded arrays/text, project-relative paths, deterministic canonical JSON.

v1

Transaction safety

Fingerprint before staging, copy without symlink following, verify staged bytes, atomic rename, rollback on failure.

32 TESTS

Engine trust gate

Strict JSON, bundle containment, regular-file checks, SHA-256 replay, exact glTF candidate selection.

2/2 CTEST

Problem solved: timestamp checks and ad-hoc exporters cannot prove that the engine is loading the artist-approved version. Content identity and an immutable contract can.

[KairoPipelineCore](#)[KairoAssets](#)[KairoGameEngine](#)

PROJECT 02 / HOST-NEUTRAL VERIFIED

Houdini CacheGuard

A cache assistant for the expensive moment before, during, and after simulation. It turns a failed sequence into an exact resume plan instead of another full recook.

1

BEFORE COOK

Validate \$F tokens, project containment, frame range, free space, and cache budget.

2

AFTER INTERRUPTION

Scan the directory and compress missing frames into resumable ranges.

3

BEFORE PUBLISH

Compare upstream graph fingerprint, reject stale or incomplete frames.

4

AT HANDOFF

Publish every cache frame with provenance and immutable SHA-256 identity.

Failure made actionable

Example: frames 1005-1008 missing; cache changed upstream; estimated completion exceeds configured free-space reserve.

Verification

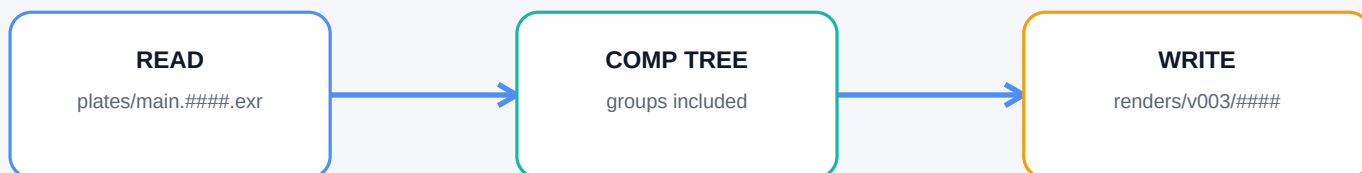
10 unit tests plus compile, JSON package validation, shelf XML validation, and green GitHub CI. Native HOM/hython is license-gated.

[GitHub: KairoHoudini](#)

PROJECT 03 / HOST-NEUTRAL VERIFIED

Nuke ShotDoctor

A Read/Write preflight that catches plate, range, colorspace, connection, path, and overwrite errors before render submission.

**NUKE_READ_FRAMES_MISSING**

Frames 1007-1010 are absent from the plate.

NUKE_READ_COLORSPACE

Read colorspace is outside the approved show profile.

NUKE_WRITE_DISCONNECTED

The render output has no input branch.

NUKE_WRITE_OVERWRITE

14 frames already exist in this version.

Cross-DCC trust

Load Houdini cache or render manifests, check allowed kind, output presence, byte count, and SHA-256 before ingest.

Verification

10 tests cover rules, grouped-node adapter shape, controlled no-host behavior, valid ingest, missing output, and mutation rejection. CI is green.

[GitHub: KairoNuke](#)

PROJECT 04 / HOST-NEUTRAL VERIFIED

Maya SceneDoctor

A dockable scene handoff panel backed by Maya API 2.0 mesh inspection and command-layer project diagnostics.

REFERENCES

Missing/unloaded files, external paths, duplicate or reserved namespaces.

TRANSFORMS

Dirty export roots and zero/negative scale without silent artistic fixes.

GEOMETRY

Empty meshes, missing UVs/materials, nonmanifold edges, lamina faces.

TEXTURES

Missing/external files and colorspace outside the project profile.

DOCKABLE ARTIST LOOP

OPEN PANEL -> VALIDATE SCENE -> DOUBLE-CLICK DIAGNOSTIC -> FIX IN CONTEXT -> REVALIDATE

Once preflight is clear, the package publisher stages the .ma/.mb scene, project references, and textures, verifies them, and exposes the version atomically.

8 TESTS

API 2.0 ADAPTER

ATOMIC PACKAGE

GREEN CI

Native Maya panel/API execution remains an explicit install and license gate.

[GitHub: KairoMaya](#)

APPLICATION SUMMARY

Production engineering in service of artists

This suite is a solo engineering portfolio built around an artist review loop: visible diagnostics, navigable failures, bounded fixes, repeatable demos, and explicit host gates. It is ready for collaborative usability testing rather than presented as unverified group work.

SURFACE	EVIDENCE	STATUS
Pipeline core	32 tests / multi-platform CI	GREEN
Blender	8 tests / native host + render	NATIVE
Houdini	10 tests / package + CI	HOST GATE
Nuke	10 tests / adapter + CI	HOST GATE
Maya	8 tests / publisher + CI	HOST GATE
KairoAssets	C++23 build / 2 of 2 CTest	NATIVE

Python

Shared package plus four DCC adapters. Pure rules are isolated from host APIs for fast feedback and CI.

C++23

Strict engine-side consumer integrated into KairoAssets and pinned in KairoGameEngine.

Problem solving

Late failures become precise actions: resume ranges, path fixes, integrity rejection, and rollback.

REPOSITORY INDEX

Portfolio hub

Core

Blender

Houdini

Nuke

Maya

C++ engine assets

Prepared for UTS Master of Animation and Visualisation